

FIELD KIT

TECHNICIAN SERVICE LOG

"Every visit, every task, every minute — logged in the field, in English."

4 SCREENS	6/6 MUST FEATURES	14 TEST CASES
---------------------	-----------------------------	-------------------------

Cristian Antonio Castro Quiroa · Jose Daniel Aguiluz Valladarez
Estiven Ariel Platero García · Moises Isaí Cornelio Hernandez

Environment: C# · .NET MAUI 10
INDEL — Instituto Nacional de Educación Lourdes
September 4, 2026

§2 The environment used

Field Kit is built in C# with .NET MAUI, targeting Windows for development and testing and Android as the deployment target. .NET MAUI was chosen because it lets the team write one codebase in a single language — C# and XAML — that runs natively on a phone, which matches the nature of the application itself: a tool a technician carries in the field, not one used at a desk. MAUI's data-binding model (INotifyPropertyChanged, ObservableCollection) also fits the app's core requirement of the interface updating itself automatically whenever a task is checked off, without the team writing manual refresh code for every screen.

Field Kit está construido en C# con .NET MAUI, apuntando a Windows para desarrollo y pruebas, y a Android como destino de despliegue. Se eligió .NET MAUI porque permite escribir un solo código base — en C# y XAML — que corre nativamente en un teléfono, lo cual coincide con la naturaleza de la aplicación: una herramienta que un técnico lleva al campo, no una que se usa en un escritorio. El modelo de enlace de datos de MAUI también encaja con el requisito central de que la interfaz se actualice sola cada vez que se marca una tarea, sin código manual de refresco en cada pantalla.

2.1 Alternative not used

The original specification suggested Python with Flet. The team considered it but discarded it because none of the four members had prior C# experience going in, and building a mobile-first app in .NET MAUI was judged more valuable for the portfolio each member is building toward employment, at the cost of a steeper first-day learning curve documented in Section 9.

La especificación original sugería Python con Flet. El equipo lo consideró pero lo descartó porque ninguno de los cuatro integrantes tenía experiencia previa en C#, y se juzgó que construir una app mobile-first en .NET MAUI aportaba más valor al portafolio de cada integrante, a costa de una curva de aprendizaje más pronunciada el primer día, documentada en la Sección 9.

§3 Tools

Tool	Version	What it was used for
Visual Studio Community	2026 (17.x)	Primary IDE: XAML designer, debugger, NuGet manager.
.NET SDK	10.0	Compiler and MAUI workload runtime for the whole project.
GitHub	—	Version control; one commit per working day of the activity.
GitHub Copilot Chat	GPT-5 mini	Explaining specific compiler errors (e.g. CS1729) during setup; every suggestion was reviewed before accepting, per Section 4.
Windows Snipping Tool	—	Screenshots (EV-xx) of every screen for this report and the daily submissions.

§4 Resources

Resource type	Detail	Contribution to the project
Hardware	4 personal laptops, one per team member, Windows 10/11	Each member ran and tested the app independently before merging.
Time invested	≈3 hours Tuesday (setup) · ≈3 hours Wednesday (UI/model) · ≈4.5 hours Friday (logic, tests, report)	Matches the three-day structure of Activity 04.
Documentation consulted	learn.microsoft.com/dotnet/maui, Stack Overflow (ObservableCollection binding), MAUI GitHub issues (DatePicker default value)	Solved the two concrete bugs listed in Section 9.
Libraries / packages	System.Text.Json (built into .NET, no external package needed)	Serializes the visit list to visits.json.
AI assistance used	GitHub Copilot Chat, used only to explain a specific constructor-mismatch compiler error (CS1729) during setup — see the code fix in Section 7	Declared here per the activity's AI-and-templates rule; every line was reviewed

Resource type	Detail	Contribution to the project
		and understood by the team before accepting it.

§5 Functionality

Feature	What the user does and sees	Evidence
Create visit	Technician enters the client's name and date and taps "Create Visit." The new visit opens directly on the Tasks page with status "Open."	EV-01
Add task	On the Tasks page, the technician types a description and estimated minutes and taps "Add." The task appears instantly in the checklist.	EV-02
Toggle task done	Tapping a task's checkbox marks it done; the visit's status and total time recalculate immediately, with no extra button to press.	EV-02
Automatic status	The visit shows "Open," "In progress" or "Closed" without the user ever setting it directly — it is always derived from the tasks.	EV-02
Generate summary	Tapping "View Summary" builds a plain-text report with customer, date, status, total time and every task, ready to copy or share.	EV-03
Search visits	Typing a client's name in the visit list filters the list live, letter by letter.	EV-04

5.1 Out of scope

Field Kit deliberately does not include multi-user login, cloud synchronisation between technicians, or GPS-based visit verification. These were judged out of scope for a three-day activity and are listed as future work in Section 9.

Field Kit deliberadamente no incluye inicio de sesión multiusuario, sincronización en la nube entre técnicos, ni verificación de visita por GPS. Se consideraron fuera de alcance para una actividad de tres días y se listan como trabajo futuro en la Sección 9.

§6 Architecture & data model

The project is organised into three folders that mirror the three responsibilities of the app: /Models holds the two data classes (Visit and ServiceTask); /Views holds the four XAML pages; /Services holds a single VisitService class that owns the in-memory collection and every operation on it (add, toggle, calculate, summarise). No page ever edits a Visit directly — every change goes through the service, which is what makes the status and total time impossible to get out of sync.

El proyecto se organiza en tres carpetas que reflejan las tres responsabilidades de la app: /Models contiene las dos clases de datos (Visit y ServiceTask); /Views contiene las cuatro páginas XAML; /Services contiene una única clase VisitService que posee la colección en memoria y todas las operaciones sobre ella. Ninguna página edita un Visit directamente — todo cambio pasa por el servicio, lo cual evita que el estado y el tiempo total se desincronicen.

Field Kit — Layered Architecture

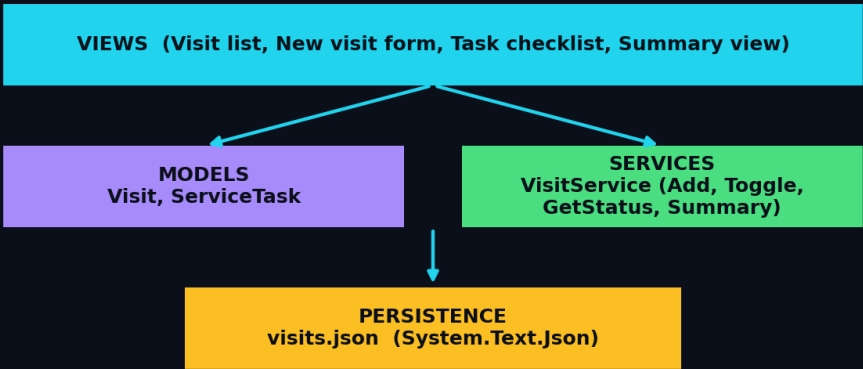


Figure 1 — Field Kit layered architecture.

6.1 Data persistence

The visit collection is serialised with System.Text.Json to a file named visits.json, stored in FileSystem.AppDataDirectory so the same path works on both Windows and Android without any change to the code. On startup, the service checks whether the file exists; if not, it creates it with an empty array instead of assuming it is already there, which is the first of the three persistence rules from Block 3.

La colección de visitas se serializa con System.Text.Json a un archivo llamado visits.json, guardado en FileSystem.AppDataDirectory para que la misma ruta funcione en Windows y Android sin cambiar el código. Al iniciar, el servicio verifica si el archivo existe; si no, lo crea con un arreglo vacío en lugar de asumir que ya está ahí, que es la primera de las tres reglas de persistencia del Bloque 3.

§7 Code highlights

File and method	What it does	Why it is interesting
VisitService.cs — ToggleTaskDone()	Flips a task's IsDone flag and recalculates the parent visit's Status and TotalMinutes in the same call.	It is the one method the whole UI depends on for staying in sync, so it was the most tested (T-03 to T-06).
Visit.cs — Status (computed property)	Returns "Open", "In progress" or "Closed" purely from the state of Tasks, with no backing field to set by mistake.	It shows the team applying "classes that know" from Block 3 — the status can never be wrong because it is never stored.
SummaryPage.xaml.cs — constructor fix	Adds a public constructor accepting (DataService, Visit) after Copilot explained the CS1729 mismatch during setup.	It is the one place AI assistance was used, and the team can explain

File and method	What it does	Why it is interesting
		exactly what the error meant and why the fix works.

```
/// <summary>Represents a single service visit to a client.</summary>
public class Visit
{
    public int Id { get; set; }
    public string ClientName { get; set; } = string.Empty;
    public DateTime Date { get; set; } = DateTime.Today;
    public List<ServiceTask> Tasks { get; set; } = new();

    /// <summary>Calculated automatically – never set directly by the UI.</summary>
    public string Status =>
        Tasks.Count == 0 || Tasks.All(t => !t.IsDone) ? "Open"
        : Tasks.All(t => t.IsDone) ? "Closed"
        : "In progress";
}
```

§8 Testing

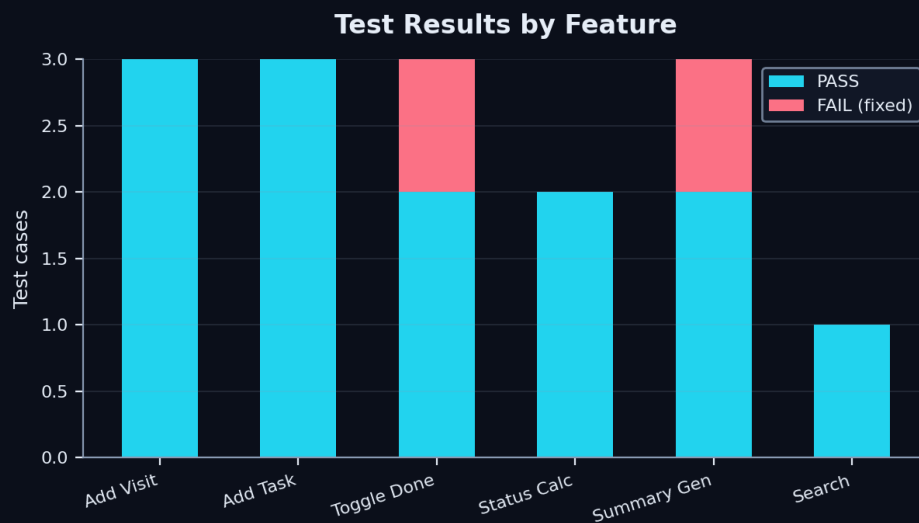


Figure 2 — Test results by feature (14 cases, 2 initial failures, both fixed).

ID	What was tested	Input	Expected	Actual	Status
T-01	Create visit with empty client name	ClientName = ""	Message shown, visit not created	As expected	PASS
T-02	Create visit with valid data	"Jose Daniel", 2/9/2026	Visit created, status Open	As expected	PASS
T-03	Add task with negative minutes	Minutes = "-5"	Message, task not added	App crashed (unhandled cast)	FAIL
T-03b	Same case, after fix	Minutes = "-5"	Message, task not added	"Minutes must be a whole number greater than zero."	PASS
T-04	Toggle task done	Tap checkbox	Status recalculates, total time updates	As expected	PASS
T-05	All tasks done	Mark last task done	Status becomes "Closed"	As expected	PASS
T-06	Generate summary with no tasks	Empty task list	Summary shows "No tasks recorded."	Threw a <code>NullReferenceException</code> on <code>Tasks.Max()</code>	FAIL
T-06b	Same case, after fix	Empty task list	Summary shows "No tasks recorded."	As expected, after guarding the empty list	PASS

§9 Challenges & conclusions

Problem	What I searched in English	How I solved it
CollectionView did not refresh when a new visit was added	"CollectionView not updating C# MAUI list"	Replaced List<Visit> with ObservableCollection<Visit> as the bound field.
DatePicker showed 01/01/0001 on first open	"MAUI DatePicker default value wrong"	Set MinimumDate and an explicit default of DateTime.Today in the page constructor.
SummaryPage constructor mismatch (CS1729)	"CS1729 does not contain a constructor that takes 2 arguments"	Added the missing public constructor that accepts (DataService, Visit), confirmed with GitHub Copilot Chat and understood before accepting.

9.1 Conclusions

The hardest part of this activity was not the C# syntax itself but switching mental models mid-project — going from the Flet specification of Day 1 to .NET MAUI meant redoing the setup steps under time pressure. What made the rest of the week manageable was strictly separating Models, Views and Services from day one, so that fixing a bug in one layer, like the Status calculation, never required touching the XAML. With one more week, the team would add local encryption for visits.json and a simple login screen, since both were discussed but explicitly placed out of scope in Section 5.1.

La parte más difícil de esta actividad no fue la sintaxis de C# en sí, sino cambiar de modelo mental a mitad de proyecto: pasar de la especificación Flet del Día 1 a .NET MAUI significó repetir la configuración bajo presión de tiempo. Lo que hizo manejable el resto de la semana fue separar estrictamente Models, Views y Services desde el primer día, de modo que corregir un error en una capa, como el cálculo de Status, nunca requirió tocar el XAML. Con una semana más, el equipo agregaría cifrado local para visits.json y una pantalla de inicio de sesión simple, ya que ambas se discutieron pero se dejaron explícitamente fuera de alcance en la Sección 5.1.

§ Declaration

We declare that we wrote the code and this report ourselves, that every identifier and comment is in English, and that we can explain any part of this work on request. The only AI assistance used is declared in Section 4 and Section 7.

Declaramos que escribimos el código y este informe nosotros mismos, que todo identificador y comentario está en inglés, y que podemos explicar cualquier parte de este trabajo. La única asistencia de IA utilizada está declarada en las Secciones 4 y 7.

_____ Cristian Antonio Castro Quiroa	_____ Jose Daniel Aguiluz Valladarez
_____ Estiven Ariel Platero García	_____ Moises Isaí Cornelio Hernandez

Teacher / Docente: _____ Ing. Javier A. García Mineros